

Beginner+ Full-Stack Task

E-Commerce Application — Go + PostgreSQL + Redis + React

1. Görevin Amacı

Gerçek bir web uygulamasının temel parçalarını öğrenmek için full-stack bir e-commerce uygulaması geliştir.

Daha önce Go kullanarak:

- UDP protocol
- Terminal chat application
- Bubble Tea UI
- Encryption
- Rooms
- Client/server communication

içeren bir uygulama geliştirdin.

Bu projede artık web application tarafına geçerek aşağıdaki konuları öğrenmen beklenmektedir:

- HTTP
- REST API
- Go backend architecture
- PostgreSQL
- SQL
- Database transactions
- Redis
- Authentication
- Authorization
- Sessions
- Caching
- Cache invalidation
- Background queues
- Workers
- Email delivery
- React
- Tailwind CSS
- Admin panel
- File uploads
- Image optimization
- Video optimization
- FFmpeg
- Multi-language
- Manual bank transfer payments

- Payment approval/rejection
- Docker
- Makefile
- Automated testing
- Code coverage

Bu proje bir Amazon clone olmak zorunda değildir.

Ama ana akışların tamamı gerçek şekilde çalışmalıdır.

2. Kullanılacak Teknolojiler

Backend

- Go 1.26.5 
- PostgreSQL 18.4
- Redis 8.8.x
- `pgx`
- `pgxpool`
- `go-redis`
- REST API

Mümkün olduğunca standart Go packages kullan.

Çok ağır bir backend framework kullanmak zorunlu değildir.

Frontend

- React 19.2.x
- Vite
- Tailwind CSS 4.3.x
- React Router

Testing için:

- Vitest
- React Testing Library

kullanılabilir.

Infrastructure

- Docker
- Docker Compose
- Makefile
- Mailpit

- FFmpeg

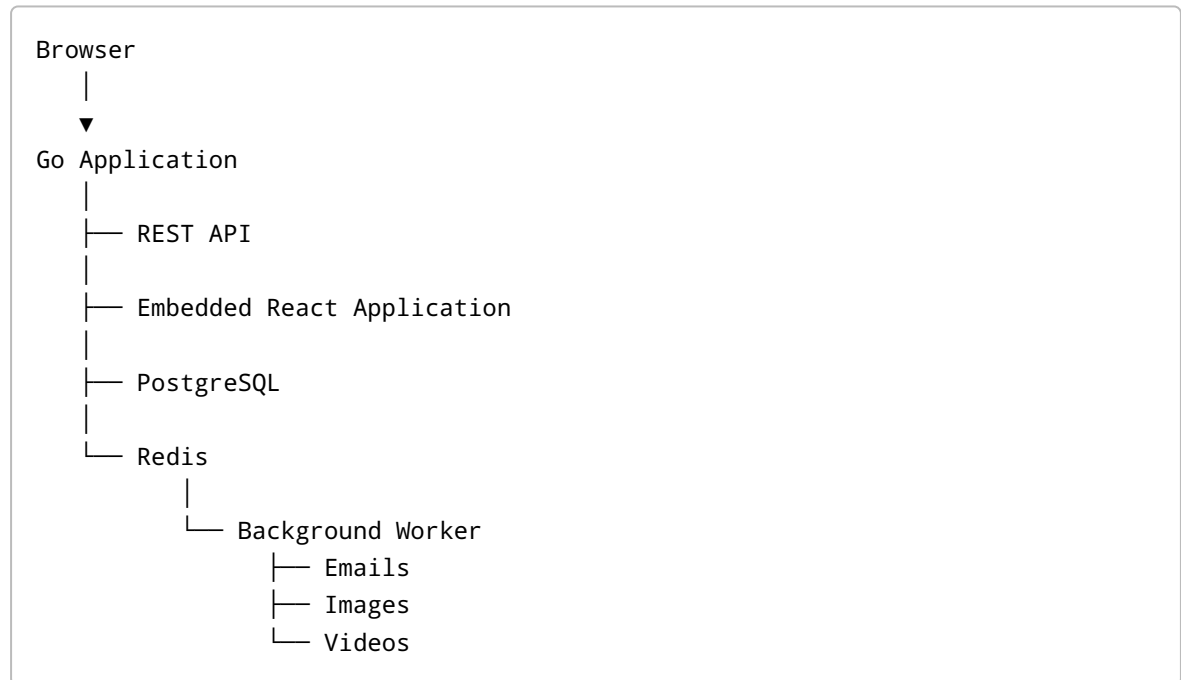
Opsiyonel E2E test için:

- Playwright

kullanılabilir.

3. Ana Mimari

Uygulama production ortamında aşağıdaki şekilde çalışmalıdır:



4. React Ayrı Server Olmayacak

Production'da React için ayrı:

Node.js server
Vite dev server
Nginx
React container

kullanılmamalıdır.

React build edilmelidir:

React Source



Vite Build



dist/



Go embed



Go Binary

Go:

```
//go:embed web/dist/*  
var frontend embed.FS
```

benzeri bir yapı ile React build'ini binary içine almalıdır.

Sonuç:

GET /api/products

→ Go REST API

GET /products

→ React

GET /admin/products

→ React

GET /my-orders/123

→ React

olmalıdır.

React Router SPA fallback desteklenmelidir.

Kullanıcı doğrudan:

```
http://localhost:8080/admin/payments
```

adresine giderse 404 almamalıdır.

5. Repository Yapısı

Örnek:

```
ecommerce/
├── cmd/
│   └── ecommerce/
│       └── main.go
├── internal/
│   ├── auth/
│   ├── users/
│   ├── categories/
│   ├── products/
│   ├── cart/
│   ├── orders/
│   ├── payments/
│   ├── admin/
│   ├── database/
│   ├── cache/
│   ├── queue/
│   ├── mail/
│   ├── media/
│   ├── middleware/
│   ├── config/
│   └── server/
├── migrations/
├── web/
│   ├── src/
│   ├── public/
│   ├── package.json
│   └── vite.config.js
├── uploads/
├── Dockerfile
├── docker-compose.yml
├── Makefile
├── .env.example
├── .gitignore
├── go.mod
├── go.sum
└── README.md
```

Birebir aynı yapı zorunlu değildir.

Ancak bütün uygulama tek bir:

```
main.go
```

dosyasına yazılmamalıdır.

6. Go Binary

Tercihen tek Go binary kullanılmalıdır.

Örneğin:

```
./ecommerce server
```

HTTP server çalıştırır.

```
./ecommerce worker
```

background worker çalıştırır.

```
./ecommerce migrate
```

migration çalıştırır.

```
./ecommerce migrate-down
```

migration rollback yapar.

```
./ecommerce create-admin
```

admin oluşturur.

```
./ecommerce seed
```

development seed data oluşturur.

7. Makefile Zorunludur

Repository içerisinde mutlaka:

Makefile

bulunmalıdır.

Makefile normal developer workflow'un ana giriş noktası olacaktır.

Uzun Docker, npm veya Go command'larını her seferinde elle yazmak yerine Makefile kullanılacaktır.

8. Minimum Makefile Targets

Minimum aşağıdaki target'lar bulunmalıdır:

```
help ✓  
  
dev ✓  
up  
down  
restart  
  
build ✓  
rebuild  
  
logs  
logs-app  
logs-worker  
  
migrate  
migrate-down  
  
create-admin  
seed  
  
frontend-install  
frontend-build  
  
test  
test-backend  
test-frontend  
test-integration  
test-e2e  
  
coverage  
coverage-backend  
coverage-frontend  
  
format  
lint
```

```
clean
```

9. Makefile — Help

Şu komut:

```
make help
```

kullanılabilir target'ları açıklamalıdır.

Örneğin:

<code>make up</code>	Start application
<code>make down</code>	Stop application
<code>make test</code>	Run all tests
<code>make coverage</code>	Generate coverage reports
<code>make migrate</code>	Run migrations
<code>make create-admin</code>	Create admin user

10. Makefile — Start Application

```
make up
```

application stack'i ayağa kaldırmalıdır.

Örneğin altında:

```
docker compose up -d --build
```

çalışabilir.

11. Makefile — Development

```
make dev
```

development ortamını çalıştırmalıdır.

Nasıl implement edeceğin sana kalmıştır.

Ancak command gerçekten kullanılabilir olmalıdır.

12. Makefile — Stop

```
make down
```



Docker services'lerini durdurmalıdır.

13. Makefile — Restart

```
make restart
```

uygulamayı yeniden başlatmalıdır.

14. Makefile — Build

```
make build
```

production application build'ini yapmalıdır.

Akış:

```
React dependencies
  ↓
React Build
  ↓
Go Build
  ↓
React embedded
  ↓
Docker Image
```

15. Makefile — Tests

```
make test
```

bütün normal test suite'i çalıştırmalıdır.

Minimum olarak:

```
Backend unit tests  
Backend integration tests  
Frontend tests
```

çalışmalıdır.

E2E tests ayrı tutulabilir.

16. Makefile — Coverage

```
make coverage
```

backend ve frontend coverage raporlarını oluşturmalıdır.

Örneğin:

```
coverage/  
├── backend/  
│   └── coverage.html  
└── frontend/  
    └── index.html
```

gibi bir output oluşturulabilir.

17. Makefile — Migration

```
make migrate
```

database migration çalıştırmalıdır.

Örneğin altında:

```
docker compose exec app ./ecommerce migrate
```

çalışabilir.

18. Makefile — Create Admin

```
make create-admin
```

admin oluşturmalıdır.

Password doğrudan Makefile içine yazılmamalıdır.

Interactive giriş tercih edilir:

```
Name:
Email:
Password:
```

19. Makefile Güvenlik Kuralları

Makefile içerisinde:

```
database password
SMTP password
admin password
session secret
API secret
```

bulunmamalıdır.

Makefile environment configuration kullanılmalıdır.

20. Kullanıcı Roller

Sistemde minimum iki role bulunmalıdır:

customer
admin

21. Customer Özellikleri

Customer:

- Register
- Verify email
- Login
- Logout
- Forgot password
- Reset password
- Profile view
- Product listing
- Product search
- Product filtering
- Product detail
- Cart
- Checkout
- Bank transfer payment
- Submit transfer notification
- View payment status
- View orders
- View order detail

yapabilmelidir.

22. Admin Özellikleri

Admin:

- Login
- Dashboard
- Product CRUD
- Category CRUD
- Stock management
- Product image upload
- Product video upload
- Order management
- Payment review
- Payment approve
- Payment reject
- User listing

yapabilmelidir.

23. PostgreSQL

Ana database PostgreSQL olacaktır.

Database connection pool kullanılmalıdır:

```
pgxpool
```

Her request geldiğinde yeni connection oluşturulmamalıdır.

24. Migration Sistemi

Database schema migration dosyaları ile yönetilmelidir.

Örnek:

```
migrations/
```

```
001_users.up.sql ✓
```

```
001_users.down.sql ✓
```

```
002_categories.up.sql ✓
```

```
002_categories.down.sql ✓
```

```
003_products.up.sql ✓
```

```
003_products.down.sql ✓
```

25. Minimum Database Tables

Minimum olarak:

```
users ✓
```

```
email_verification_tokens ✓
```

```
password_reset_tokens ✓
```

```
categories ✓
```

```
category_translations ✓
```

```
products ✓
```

```
product_translations ✓
```

product_media ✓
carts ✓
cart_items ✓
orders
order_items
payments
bank_transfer_submissions

tabloları veya eşdeğerleri bulunmalıdır.

Opsiyonel:

payment_status_history
order_status_history

26. User ✓

Minimum:

id
name
email
password_hash
role
email_verified_at
created_at
updated_at

Email unique olmalıdır.

27. Register

Endpoint:

POST /api/auth/register

Örnek request:

```
{  
  "name": "John Doe",  
  "email": "john@example.com",  
  "password": "secure-password"  
}
```

Akış:

```
Validate ✓  
↓  
Check duplicate email ✓  
↓  
Hash password ✓  
↓  
Create user ✓  
↓  
Create verification token ✓  
↓  
Queue verification email  
↓  
Return response ✓
```

28. Password Hashing

Password plain text olarak tutulmamalıdır.

Tercihen:

Argon2id ✓

kullan.

Normal:

MD5
SHA1
SHA256

password hashing yerine kullanılmamalıdır.

29. Email Verification

Register sonrasında email verification gereklidir.

Token:

- Random
- Secure
- Expiring
- Single use

olmalıdır.

Email HTTP request içerisinde gönderilmemelidir.

30. Email Queue

Akış:

```
HTTP Request
↓
Create email job
↓
Redis
↓
HTTP response

Worker
↓
Redis job
↓
SMTP
↓
Mailpit
```

31. Login

Endpoint:

```
POST /api/auth/login
```

Başarılı login sonrasında Redis session oluşturulmalıdır.

32. Redis Session

Örneğin:

```
session:40f8a...
```

Value:

```
{  
  "user_id": 123  
}
```

TTL olmalıdır.

33. Authentication Cookie

Cookie:

```
HttpOnly  
Secure  
SameSite  
Path  
MaxAge / Expires
```

desteklemelidir.

Session/token:

```
localStorage
```

içerisinde tutulmamalıdır.

34. Current User

Endpoint:

```
GET /api/auth/me
```

olmalıdır.

35. Logout

POST /api/auth/logout

yapıldığında:

Delete Redis session
↓
Clear browser cookie

yapılmalıdır.

36. Forgot Password

POST /api/auth/forgot-password

akışı:

Email
↓
Reset token
↓
Email queue
↓
Worker
↓
Reset email

37. Reset Password

POST /api/auth/reset-password

Token:

- Expire olmalı
 - Single use olmalı
 - Başarılı reset sonrası **invalidate** olmalı
-

38. Authorization

Admin API:

```
/api/admin/*
```

altında tutulabilir.

Customer admin endpoint'ine erişirse:

```
403 Forbidden
```

dönmelidir.

Bu kontrol backend'de yapılmalıdır.

39. Middleware

Authentication middleware akışı:

```
Request
↓
Read Cookie
↓
Read Redis Session
↓
Find User
↓
Attach User to Context
↓
Handler
```

Admin authorization için ayrıca middleware kullanılabilir.

40. Categories

Admin category CRUD yapabilmelidir.

Örneğin:

```
GET    /api/categories

POST   /api/admin/categories
PUT    /api/admin/categories/:id
DELETE /api/admin/categories/:id
```

41. Multi-Language Category

Category:

```
categories
```

ve:

```
category_translations
```

olarak ayrılabilir.

Örneğin:

```
locale = en
name = Electronics
```

```
locale = tr
name = Elektronik
```

42. Products

Product minimum:

```
id
category_id
slug
price
stock
status
created_at
updated_at
```

Status:

```
draft
active
archived
```

43. Product Translation

Translation:

```
product_id
locale
name
description
```

Minimum locale:

```
en
tr
```

44. Money

Money için doğrudan float kullanımından kaçın.

Tercihen:

```
PostgreSQL NUMERIC
```

veya integer minor units kullan.

Örneğin:

```
12990
```

=

```
129.90 TRY
```

Seçilen yöntem README'de açıklanmalıdır.

45. Product Listing

Endpoint:

```
GET /api/products
```

Minimum:

- Pagination
- Search
- Category filter
- Min price
- Max price
- Sort

desteklemelidir.

Örnek:

```
/api/products?  
page=1&limit=20&search=keyboard&category=electronics&sort=price_asc
```

46. Pagination Response

Örneğin:

```
{  
  "data": [],  
  "pagination": {  
    "page": 1,  
    "limit": 20,  
    "total": 140,  
    "pages": 7  
  }  
}
```

47. Product Detail

Endpoint:

```
GET /api/products/:slug
```

Frontend:

```
/products/:slug
```

Göster:

- Name
- Description
- Price
- Stock
- Category
- Images
- Video
- Add To Cart

48. Redis Product Cache

Product detail Redis'te cache edilmelidir.

Akış:

```
GET Product
↓
Redis
↓
Cache hit?
├─ Yes → return
└─ No
    ↓
    PostgreSQL
    ↓
    Cache result
    ↓
    return
```

49. Cache TTL

Cache TTL kullanılmalıdır.

Örneğin:

5-15 dakika

uygundur.

50. Cache Invalidation

Admin:

Product Update
Price Update
Stock Update
Archive
Delete

yaptığında ilgili cache temizlenmelidir.

TTL dolmasını beklemek yeterli değildir.

51. Cart

Endpoint örnekleri:

```
GET    /api/cart
POST   /api/cart/items
PUT    /api/cart/items/:id
DELETE /api/cart/items/:id
```

52. Cart Validation

Backend:

quantity > 0

kontrol etmelidir.

Stock kontrol edilmelidir.

Frontend'den gelen price veya total değerine güvenilmemelidir.

53. Checkout

Checkout endpoint:

```
POST /api/orders
```

olabilir.

Checkout sırasında gerçek payment provider kullanılmayacaktır.

Payment method:

```
bank_transfer
```

olacaktır.

54. Checkout Transaction

Checkout PostgreSQL transaction içerisinde yapılmalıdır.

```
BEGIN
↓
Read cart
↓
Read products
↓
Validate stock
↓
Calculate total
↓
Create order
↓
Create order items
↓
Create payment
↓
Reduce stock
↓
COMMIT
```

Hata:

ROLLBACK

55. Concurrent Stock

İki kullanıcı aynı anda son ürünü satın almaya çalıştığında stok negatif olmamalıdır.

Bu problem transaction/locking ile düzgün çözülmelidir.

56. Order

Minimum:

```
id
user_id
status
total_amount
currency
created_at
updated_at
```

Order status:

```
pending_payment
payment_review
paid
processing
shipped
completed
cancelled
```

57. Order Item Snapshot

Order item üzerinde:

```
product_id
product_name
quantity
```

```
unit_price
total_price
```

snapshot olarak tutulmalıdır.

Ürün fiyatı sonradan değişirse geçmiş sipariş değişmemelidir.

58. Bank Transfer Payment

Checkout sonrasında customer'a banka bilgileri gösterilmelidir.

Örneğin:

```
Bank:
Example Bank

Account Holder:
Example Commerce Ltd.

IBAN:
TR00 0000 0000 0000 0000 0000 00

Amount:
1,250.00 TRY

Reference:
ORD-X9K24P
```

59. Bank Configuration

Bank bilgileri environment veya configuration üzerinden gelmelidir.

Örneğin:

```
BANK_NAME
BANK_ACCOUNT_NAME
BANK_IBAN
BANK_CURRENCY
```

UI component içine hard-code edilmemelidir.

60. Payment Reference

Her order için unique reference code oluştur.

Örneğin:

```
ORD-X9K24P
```

Reference database üzerinde unique olmalıdır.

61. Payment

Minimum:

```
id
order_id
user_id
method
amount
currency
reference_code
status
reviewed_by
reviewed_at
review_note
created_at
updated_at
```

62. Payment Status

Status:

```
awaiting_transfer
submitted
approved
rejected
cancelled
```

63. Payment ve Order Ayırır

Payment status:

submitted

ise order:

payment_review

olabilir.

Payment:

approved

olursa order:

paid

olmalıdır.

User'ın:

I made the transfer

demesi payment'in approved olduğu anlamına gelmez.

64. Order Created Email

Checkout sonrası email queue'ya:

send_bank_transfer_instructions

job ekle.

Email:

- Order number
- Amount

- Currency
- Bank
- Account holder
- IBAN
- Reference code

içermelidir.

65. Transfer Submission

Customer order detail sayfasında:

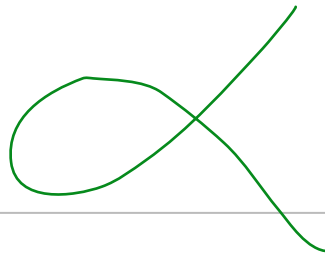
I made the transfer

butonuna basabilmelidir.

66. Transfer Submission Form

Minimum:

Reference Code
Transfer Date
Transferred Amount
Sender Name
Note



alanları bulunmalıdır.

67. Transfer Submission Endpoint

Örneğin:

POST /api/orders/:id/payment/bank-transfer

Request:

```
{  
  "reference_code": "ORD-X9K24P",  
  "transfer_date": "2026-08-07",  
  "amount": "1250.00",  
  "sender_name": "John Doe",  
}
```

```
"note": "Transfer completed."  
}
```

68. Transfer Submission Validation

Backend kontrol etmelidir:

- Order current user'a mı ait?
- Payment bank transfer mı?
- Reference doğru mu?
- Payment approved olmuş mu?
- Payment zaten review altında mı?
- Order cancelled mı?
- Amount format geçerli mi?
- Date format geçerli mi?

69. Submission Sonrası

Customer transfer bildirdiğinde:

```
Payment = submitted  
Order = payment_review
```

olmalıdır.

Payment otomatik approved yapılmamalıdır.

70. Transfer Submitted Email

Customer'a:

```
We received your transfer notification.  
Your payment is waiting for review.
```

email'i gönderilmelidir.

Admin notification email de gönderilebilir.

Bütün email'ler queue üzerinden gitmelidir.

71. Admin Payments

Frontend:

/admin/payments

sayfası oluřtur.

Göster:

- Order
- Customer
- Amount
- Expected amount
- Reference
- Sender
- Transfer date
- Submitted date
- Status

72. Payment Filters

Admin:

Submitted
Approved
Rejected

statuslarına göre filtreleyebilmelidir.

73. Admin Payment Detail

Admin:

/admin/payments/:id

üzerinde:

- Customer
- Order
- Order items
- Expected amount

- Submitted amount
- Reference
- Transfer date
- Sender
- Customer note
- Current status

görmelidir.

74. Payment Approve

Endpoint:

```
POST /api/admin/payments/:id/approve
```

Admin approve yaptığında:

```
Payment = approved  
Order = paid
```

olmalıdır.

Ayrıca:

```
reviewed_by  
reviewed_at  
review_note
```

kaydedilmelidir.

75. Payment Approval Email

Payment approve edilince customer'a:

```
Payment Approved
```

email'i queue üzerinden gönderilmelidir.

76. Payment Reject

Endpoint:

```
POST /api/admin/payments/:id/reject
```

Request:

```
{  
  "reason": "Transferred amount does not match the order total."  
}
```

Reason zorunlu olmalıdır.

77. Payment Reject Sonrası

Payment:

```
rejected
```

olmalıdır.

Order tekrar:

```
pending_payment
```

durumuna geçebilir.

Customer yeni transfer submission yapabilmelidir.

78. Payment Rejection Email

Customer'a:

```
Payment Rejected  
  
Reason:  
Transferred amount does not match the order total.
```

email'i gönderilmelidir.

Redis email queue kullanılmalıdır.

79. Minimum Email Türleri

Minimum email job'ları:

```
send_verification_email  
  
send_password_reset_email  
  
send_bank_transfer_instructions  
  
send_transfer_submission_received  
  
send_payment_approved  
  
send_payment_rejected  
  
send_order_status_changed
```

80. Stock Policy

Beginner task için:

```
Order oluşturulduğunda stock azalt.
```

Order tamamen cancel edilirse stock geri eklenebilir.

Bu davranış transaction ile yapılmalıdır.

README içerisinde stock yaklaşımını açıkla.

81. Admin Order Management

Admin paid order'ı:

```
paid  
↓  
processing  
↓  
shipped
```

↓
completed

durumlarına geçirebilmelidir.

Henüz payment approved olmayan order:

shipped

yapılamamalıdır.

82. Admin Dashboard

Minimum cards:

Total Users
Total Products
Total Orders
Approved Sales Total
Payments Waiting Review

submitted payment'lar sales total'a dahil edilmemelidir.

83. Product Image Upload

Admin multiple image upload yapabilmelidir.

Minimum:

JPEG
PNG
WebP

destekle.

Kontrol:

- MIME
- Maximum size
- Safe filename
- Random storage name

84. Image Processing Queue

Akış:

```
Upload
↓
Save Original
↓
Create Media DB Record
↓
Redis Image Queue
↓
HTTP Response

Worker
↓
Resize
↓
Optimize
↓
Generate WebP
↓
Generate Thumbnails
↓
Update Media
```

85. Image Outputs

Örneğin:

```
320.webp
640.webp
1280.webp
```

oluştur.

86. Video Upload

Admin product için video yükleyebilmelidir.

Video processing HTTP upload request içerisinde yapılmamalıdır.

87. Video Queue

Akış:

```
Upload
↓
Save original
↓
Create media record
↓
Redis video job
↓
Return response
```

Worker:

```
FFmpeg
↓
Optimize
↓
MP4
↓
Thumbnail
↓
Update media status
```

88. Media Status

```
pending
processing
ready
failed
```

statusları kullanılabilir. 

89. Redis Queues

Minimum:

```
email
image
video
```

queue bulunmalıdır.

Basit Redis List + blocking pop yeterlidir.

Redis Streams bonus olarak kullanılabilir.

90. Queue Retry

Job hata verirse retry yapılmalıdır.

Örneğin maksimum:

```
3 attempts
```

Son attempt başarısızsa loglanmalıdır.

91. Worker

Docker Compose'da:

```
app
worker
```

aynı Go binary/image kullanılmalıdır.

Örneğin:

```
app:
  ecommerce server

worker:
  ecommerce worker
```

92. Mailpit

Development email için Mailpit kullanılmalıdır.

SMTP:

mailpit:1025

UI:

http://localhost:8025

93. Internationalization

Minimum:

English
Turkish

destekle.

UI translation:

web/src/locales/en.json
web/src/locales/tr.json

94. Product Translation

Product içerikleri de multilingual olmalıdır.

product_translations

tablosu kullanılabilir.

95. React Pages

Minimum:

Home

Products

Product Detail

Login

Register

Verify Email

Forgot Password

Reset Password

Cart

Checkout

Profile

My Orders

Order Detail

Bank Transfer Submission

Admin Dashboard

Admin Products

Admin Product Create

Admin Product Edit

Admin Categories

Admin Orders

Admin Order Detail

Admin Payments

Admin Payment Detail

Admin Users

96. React Components

Reusable components oluştur.

Örneğin:

Button

Input

Select

Modal

Alert

Spinner

Navbar
Footer

ProductCard
ProductGrid
Pagination

Table
FileUpload

LanguageSelector

PaymentStatusBadge
OrderStatusBadge

97. React API Layer

API logic'i componentlere dağıtma.

Örneğin:

```
src/api/  
  
client.js  
auth.js  
products.js  
cart.js  
orders.js  
payments.js  
admin.js
```

98. Frontend State

API işlemlerinde minimum:

```
loading  
success  
error
```

state düşünülmelidir.

Double submission engellenmelidir.

Özellikle:

```
Checkout
Transfer Submission
Payment Approve
Payment Reject
```

işlemlerinde button processing sırasında disable edilmelidir.

99. Validation

Backend mutlaka validate etmelidir.

Minimum:

```
Email
Password
Name

Price
Stock

Quantity

Category
Product status

Payment status
Reference code
Transfer amount
Transfer date

Image type
Image size

Video type
Video size

Pagination
Sort
```

Frontend validation tek başına yeterli değildir.

100. API Response Format

Başarı:

```
{
  "data": {
    "id": 123
  }
}
```

Error:

```
{
  "error": {
    "code": "PAYMENT_ALREADY_APPROVED",
    "message": "Payment has already been approved."
  }
}
```

101. HTTP Status Codes

Uygun status kullan:

```
200 OK
201 Created
204 No Content

400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
409 Conflict
422 Unprocessable Entity
500 Internal Server Error
```

102. Logging

Minimum log eventleri:

```
server_started
```

```
user_registered
user_logged_in
user_logged_out

product_created
product_updated

order_created

bank_transfer_submitted
payment_approved
payment_rejected

email_job_failed
image_job_failed
video_job_failed
```

103. Sensitive Data Logging

Loglama:

```
password
password_hash
session_token
verification_token
reset_token
```

yapılmamalıdır.

104. Environment Variables

Örneğin:

```
APP_ENV
APP_URL
HTTP_ADDR

DATABASE_URL

REDIS_ADDR
REDIS_PASSWORD

SMTP_HOST
SMTP_PORT
```

SMTP_FROM

SESSION_COOKIE_NAME

SESSION_TTL

UPLOAD_PATH

MAX_IMAGE_SIZE

MAX_VIDEO_SIZE

BANK_NAME

BANK_ACCOUNT_NAME

BANK_IBAN

BANK_CURRENCY

105. .env

Repository:

.env.example

içermelidir.

Gerçek:

.env

commit edilmemelidir.

106. Docker Compose

Minimum services:

app ✓
worker
postgres
redis
mailpit



107. Docker Multi-Stage Build

Örnek:

```
Stage 1
Node
↓
React Build

Stage 2
Go
↓
Embed React
↓
Compile

Stage 3
Runtime
↓
Go Binary
↓
FFmpeg
```

108. Health Endpoint

Minimum:

```
GET /api/health
```

Örnek:

```
{
  "status": "ok"
}
```

109. Graceful Shutdown

Handle:

SIGINT
SIGTERM

Kapanırken:

HTTP server
PostgreSQL pool
Redis client

düzgün kapatılmalıdır.

110. Database Constraints

Database üzerinde sadece application validation'a güvenme.

Minimum:



```
users.email UNIQUE  
products.slug UNIQUE  
payments.reference_code UNIQUE  
UNIQUE(product_id, locale) ✗
```

gibi constraint'ler olmalıdır.

Foreign key'ler kullanılmalıdır.



111. Database Indexes

Mantıklı index'ler oluşturun.

Örneğin:

```
users.email  
products.slug  
  
orders.user_id  
  
payments.order_id  
payments.reference_code
```

```
payments.status
```

```
bank_transfer_submissions.payment_id
```

Her kolona rastgele index oluşturma.

112. TESTING — Genel Gereksinim

Testing bu görevin zorunlu bir parçasıdır.

Sadece application'ın çalışması yeterli değildir.

Critical business logic automated tests ile korunmalıdır.

113. Backend Test Türleri

Backend'de minimum:

```
Unit Tests
```

```
Service / Business Logic Tests
```

```
HTTP Handler Tests
```

```
Repository / Database Integration Tests
```

```
Redis Integration Tests
```

```
Worker Tests
```

bulunmalıdır.

114. Backend Coverage Hedefi

Backend için hedef:

```
Overall meaningful code coverage >= 80%
```

olmalıdır.

Critical business packages için:

```
>= 90%
```

hedeflenmelidir.

Özellikle:

```
auth  
cart  
orders  
payments
```

package'larında yüksek coverage beklenmektedir.

115. Coverage İçin Yanlış Yaklaşım

Sadece coverage yüzdesini artırmak için anlamsız test yazma.

Şu tarz testler yeterli değildir:

```
function çağrıldı  
error vermedi
```

Tests gerçek business behavior doğrulamalıdır.

116. Coverage Hariç Tutulabilecek Yerler

Mantıklı şekilde:

```
generated code  
  
simple main bootstrap  
  
trivial CLI wiring
```

coverage hedefinden ayrı değerlendirilebilir.

Ama business logic hariç tutulmamalıdır.

117. Backend Coverage Command

Makefile:

```
make coverage-backend
```

minimum olarak:

```
go test ./... -coverprofile=coverage.out
```

benzeri coverage üretmelidir.

HTML report üret:

```
go tool cover -html=coverage.out
```

118. Authentication Tests

Minimum:

```
Register succeeds  
  
Invalid email rejected  
  
Duplicate email rejected  
  
Password hashed  
  
Plain password not stored  
  
Verification token created  
  
Verification email queued  
  
Valid verification works  
  
Expired verification token rejected  
  
Used verification token rejected  
  
Correct login succeeds  
  
Wrong password rejected
```

Unknown user rejected

Redis session created

Logout removes session

119. Password Reset Tests

Minimum:

Forgot password creates token

Reset email queued

Expired token rejected

Invalid token rejected

Valid reset changes password

Used reset token cannot be reused

120. Authorization Tests

Minimum:

Unauthenticated user gets 401

Customer cannot access admin endpoint

Admin can access admin endpoint

Customer cannot modify another customer's order

Customer cannot access another customer's payment

121. Product Tests

Minimum:

Admin creates product

Customer cannot create product

Negative price rejected

Negative stock rejected

Invalid category rejected

Duplicate slug rejected

Product translation saved

Product cache created

Product update invalidates cache

122. Cart Tests

Minimum:

Add product

Increase quantity

Decrease quantity

Remove product

Zero quantity rejected

Negative quantity rejected

Out-of-stock quantity rejected

Cart total calculated from database price

Client supplied price ignored

123. Order Tests

Minimum:

Checkout creates order

Checkout creates order items

Checkout uses product database price

Order total calculated correctly

Stock reduced

Insufficient stock rejected

Transaction rolls back on failure

Order snapshot price stays unchanged

Cart cleared after successful checkout

124. Concurrent Stock Test

Mümkünse aynı stok için iki concurrent checkout simüle et.

Beklenen:

stock never becomes negative

Bu integration test olarak yapılabilir.

125. Payment Tests

Minimum:

Checkout creates payment

Payment method is bank_transfer

Payment starts awaiting_transfer

Payment reference is unique

Expected amount matches order

Customer can submit own payment

Customer cannot submit another user's payment

Wrong reference rejected

Approved payment cannot be resubmitted

Submitted payment enters review

Order becomes payment_review

126. Admin Payment Tests

Minimum:

Customer cannot approve payment

Customer cannot reject payment

Admin can approve submitted payment

Approve changes payment to approved

Approve changes order to paid

Approved payment cannot be approved twice

Admin can reject submitted payment

Reject requires reason

Reject stores reason

Rejected payment returns order to pending_payment

Rejected payment can be resubmitted

127. Payment Email Tests

Minimum:

Checkout queues transfer instructions email

Transfer submission queues received email

Approval queues approved email

Rejection queues rejected email

Gerçek SMTP zorunlu olarak her unit testte kullanılmamalıdır.

Queue/job oluşturulduğu doğrulanabilir.

128. Queue Tests

Minimum:

Worker reads valid job

Worker processes email job

Worker processes image job

Worker processes video job

Failed job increments attempt

Job retries

Job stops after max retries

Permanent failure logged

129. Image Worker Tests

Minimum:

Valid image accepted

Invalid file rejected

Thumbnail generated

WebP generated

Media status becomes ready

```
Processing failure sets failed
```

Image processing library behavior'unun tamamını test etmek zorunda değilsin.

Kendi business logic'ini test et.

130. Video Worker Tests

Minimum:

```
Valid video job accepted
```

```
Processing status updated
```

```
Successful processing marks ready
```

```
FFmpeg failure marks failed
```

```
Thumbnail path saved
```

FFmpeg abstraction/interface ile test edilebilir.

Her unit test sırasında gerçekten uzun video encode etmek zorunda değilsin.

131. Redis Tests

Minimum:

```
Session stored
```

```
Session expires
```

```
Logout deletes session
```

```
Product cache read
```

```
Cache miss fallback
```

```
Cache invalidation
```

```
Queue push/pop
```

132. Integration Tests

Integration tests gerçek:

```
PostgreSQL
Redis
```

ile çalışmalıdır.

Docker Compose test environment veya `testcontainers-go` kullanılabilir.

Beginner için ikisinden biri yeterlidir.

133. Frontend Tests

React tarafında:

```
Vitest
React Testing Library
```

kullanılabilir.

Minimum testler:

```
Login form validation
Register form validation
Product card rendering
Cart quantity changes
Checkout rendering
Bank transfer instructions rendering
Payment status badge
Transfer submission form
Admin payment approve button
Admin payment reject form
```

Language switch

134. Frontend Coverage Hedefi

Frontend için hedef:

`>= 70% meaningful coverage`

olmalıdır.

Critical UI logic için:

`>= 80%`

hedeflenmelidir.

Her Tailwind class'ını test etmeye çalışma.

Behavior test et.

135. Frontend Coverage Command

`make coverage-frontend`

coverage raporu üretmelidir.

136. Full Coverage Command

`make coverage`

hem:

Backend Coverage
Frontend Coverage

çalıştırılmalıdır.

Coverage threshold altında ise command failure vermelidir.

Bu özellikle önemlidir.

137. Test Command

```
make test
```

minimum:

```
Backend unit  
Backend integration  
Frontend unit/component
```

testlerini çalıştırmalıdır.

138. E2E Test

Minimum bir temel E2E happy-path test yazılması önerilir ve tercihen zorunlu kabul edilir.

Örneğin Playwright ile:

```
Register/Login  
↓  
Browse Product  
↓  
Add To Cart  
↓  
Checkout  
↓  
See Bank Transfer Reference  
↓  
Submit Transfer
```

Admin E2E:

```
Admin Login  
↓  
Open Payments  
↓  
Approve Payment
```

Tek bir tam happy path bile yeterlidir.

139. Makefile E2E

```
make test-e2e
```

E2E test suite'i çalıştırmalıdır.

140. CI Benzeri Local Command

Ek bir Makefile target oluştur:

```
make ci
```

Tercihen:

```
format check
↓
lint
↓
backend tests
↓
frontend tests
↓
coverage threshold
↓
build
```

çalıştırmalıdır.

Bu command repository push edilmeden önce çalıştırılabilir.

141. README Testing Section

README içerisinde:

```
Testing
```

bölümü olmalıdır.

Açık şekilde:

```
make test
make test-backend
make test-frontend
make test-integration
make test-e2e
make coverage
```

açıklanmalıdır.

142. README Coverage

README coverage hedeflerini açıklamalıdır:

```
Backend overall: >= 80%

Critical backend business logic:
>= 90%

Frontend:
>= 70%
```

Son mevcut coverage yüzdesini de README'de gösterebilirsin.

Örneğin:

```
Backend Coverage: 86.4%
Frontend Coverage: 74.2%
```

Fake sayı yazma.

Gerçek test output'una göre yaz.

143. README Ana Başlıkları

README minimum:

```
Project Overview

Architecture
```

Technologies

Requirements

Environment Variables

Makefile Commands

How To Run

How To Stop

Database Migrations

Create Admin

Seed Data

Authentication Flow

Payment Flow

Redis Usage

Background Jobs

Media Processing

Internationalization

Testing

Coverage

Project Structure

What I Learned

içermelidir.

144. Minimum Customer Flow

Aşağıdaki flow tamamen çalışmalıdır:

Register
↓
Verification Email Job
↓

```
Worker Sends Email
↓
Verify
↓
Login
↓
Redis Session
↓
Browse Products
↓
Search / Filter
↓
Product Detail
↓
Cart
↓
Checkout
↓
PostgreSQL Transaction
↓
Order Created
↓
Payment Created
↓
Stock Reduced
↓
Bank Transfer Instructions
↓
Email Queued
↓
Customer Makes Fake Transfer
↓
"I Made The Transfer"
↓
Submit Transfer Details
↓
Payment = submitted
↓
Order = payment_review
↓
Email Queued
↓
Wait For Admin Review
```

145. Minimum Admin Payment Flow

```
Admin Login
↓
Payments
↓
Open Submitted Payment
↓
View Order
↓
View Expected Amount
↓
View User Submitted Amount
↓
View Reference
↓
Approve / Reject
```

Approve:

```
Payment = approved
Order = paid
Email queued
```

Reject:

```
Payment = rejected
Order = pending_payment
Reason saved
Email queued
```

146. Minimum Fulfillment Flow

Payment approved olduktan sonra:

```
paid
↓
processing
↓
shipped
↓
completed
```

Admin tarafından yönetilebilmelidir.

Status değiştiğinde customer'a email gönderilebilir.

147. Minimum Media Flow

```
Admin Upload
↓
Original Saved
↓
Media = pending
↓
Redis Job
↓
Worker
↓
Media = processing
↓
Optimization
↓
Media = ready
```

Failure:

```
Media = failed
```

148. Yapılmaması Gerekenler

Aşağıdakiler kabul edilmemelidir:

- ❌ Makefile olmaması.
- ❌ Makefile target'larının sadece `echo` yapması.
- ❌ Bütün backend'in `main.go` içinde olması.
- ❌ Plain text password.
- ❌ Password için MD5/SHA256.
- ❌ Authentication token'ını localStorage'da tutmak.
- ❌ Admin security'yi sadece React'te yapmak.

- ✗ Client'tan gelen product price'a güvenmek.
- ✗ Client'tan gelen order total'a güvenmek.
- ✗ Customer'ın gönderdiği payment status'a güvenmek.
- ✗ `"approved": true` gönderince payment approve etmek.
- ✗ Transfer submit edilince otomatik payment approved yapmak.
- ✗ Customer'ın başka user'ın payment'ını submit etmesine izin vermek.
- ✗ Email'i HTTP request içerisinde göndermek.
- ✗ Image optimization'ı HTTP request içerisinde yapmak.
- ✗ FFmpeg encode'u HTTP request içerisinde yapmak.
- ✗ Secret'ları source code'a yazmak.
- ✗ Secrets'i Makefile'a yazmak.
- ✗ `.env` commit etmek.
- ✗ Production'da Vite dev server kullanmak.
- ✗ Production React için ayrı Node server kullanmak.
- ✗ React frontend için ayrı production container kullanmak.
- ✗ Cache invalidation yapmamak.
- ✗ Checkout transaction kullanmamak.
- ✗ Test yazmadan sadece manuel test yapmak.
- ✗ Coverage sayılarını fake olarak README'ye yazmak.
- ✗ Coverage yükseltmek için anlamsız tests yazmak.

149. Gereksiz Karmaşıklık Ekleme

Bu task için zorunlu değildir:

Kubernetes
Kafka

RabbitMQ
Microservices
GraphQL
gRPC
Elasticsearch
Terraform
Service Mesh
Event Sourcing
CQRS
Real Bank API
Open Banking
Credit Card Processing
Stripe
Adyen
PayPal
3D Secure
OAuth

Temel sistemi düzgün yap.

150. Bonus Özellikler

Ana task bittikten sonra:

Payment receipt upload
Payment history
Order status history
Payment expiration
Stock reservation
Admin audit log
Rate limiting
Favorites
Reviews
Coupons
Product variants
OpenAPI

MinIO / S3

Redis Streams

Dark mode

eklenebilir.

151. Beklenen Makefile Kullanımı

Normal developer workflow aşağıdaki gibi olmalıdır:

```
git clone <repo>
cd ecommerce

make up
make migrate
make create-admin
```

Tests:

```
make test
```

Coverage:

```
make coverage
```

Logs:

```
make logs
```

Stop:

```
make down
```

152. Final Delivery Checklist

Infrastructure

```
[ ] Dockerfile
[ ] Docker Compose
[ ] Makefile
[ ] .env.example
[ ] PostgreSQL
[ ] Redis
[ ] Mailpit
[ ] FFmpeg
```

Makefile

```
[ ] make help
[ ] make dev
[ ] make up
[ ] make down
[ ] make restart
[ ] make build
[ ] make rebuild
[ ] make logs
[ ] make migrate
[ ] make migrate-down
[ ] make create-admin
[ ] make seed
[ ] make frontend-install
[ ] make frontend-build
[ ] make test
[ ] make test-backend
[ ] make test-frontend
[ ] make test-integration
[ ] make test-e2e
[ ] make coverage
[ ] make coverage-backend
[ ] make coverage-frontend
[ ] make format
[ ] make lint
[ ] make ci
[ ] make clean
```

Authentication

```
[ ] Register
[ ] Email verification
```

- [] Login
- [] Redis session
- [] Secure cookie
- [] Current user
- [] Logout
- [] Forgot password
- [] Reset password

Authorization

- [] Customer role
- [] Admin role
- [] Backend middleware
- [] Resource ownership

Products

- [] Category CRUD
- [] Product CRUD
- [] Product translations
- [] Category translations
- [] Search
- [] Filters
- [] Sorting
- [] Pagination
- [] Product detail
- [] Redis cache
- [] Cache invalidation

Cart / Orders

- [] Cart
- [] Quantity validation
- [] Backend price calculation
- [] Checkout
- [] PostgreSQL transaction
- [] Stock reduction
- [] Concurrent stock safety
- [] Order items snapshot
- [] My Orders
- [] Order Detail

Payments

- [] Bank transfer payment
- [] Unique reference
- [] Bank details
- [] Payment record
- [] Payment/order state separation
- [] Transfer submission
- [] Payment submitted
- [] Admin review
- [] Payment approve
- [] Payment reject
- [] Rejection reason
- [] Re-submit rejected payment
- [] Approved payment cannot resubmit
- [] Customer cannot approve payment
- [] Customer cannot access others' payments

Email

- [] Verification email
- [] Password reset email
- [] Bank transfer instructions
- [] Transfer received
- [] Payment approved
- [] Payment rejected
- [] Order status email
- [] Redis email queue
- [] Retry

Media

- [] Multiple images
- [] Image validation
- [] Image queue
- [] Image resize
- [] WebP
- [] Thumbnail
- [] Video upload
- [] Video queue
- [] FFmpeg
- [] Video thumbnail
- [] Media status

Frontend

- ☐ React
- ☐ Tailwind CSS
- ☐ React Router
- ☐ Responsive UI
- ☐ Customer pages
- ☐ Admin pages
- ☐ API layer
- ☐ Loading states
- ☐ Error states
- ☐ Double-submit protection
- ☐ TR
- ☐ EN

Go Production Build

- ☐ React build generated
- ☐ React embedded in Go binary
- ☐ SPA fallback
- ☐ No React production server

Tests

- ☐ Backend unit tests
- ☐ Handler tests
- ☐ Service tests
- ☐ Repository integration tests
- ☐ PostgreSQL integration tests
- ☐ Redis integration tests
- ☐ Authentication tests
- ☐ Authorization tests
- ☐ Product tests
- ☐ Cache tests
- ☐ Cart tests
- ☐ Order tests
- ☐ Payment tests
- ☐ Queue tests
- ☐ Worker tests
- ☐ Email job tests
- ☐ Image worker tests
- ☐ Video worker tests
- ☐ Frontend tests
- ☐ E2E happy path

Coverage

```
[ ] Backend meaningful coverage >= 80%
[ ] Critical backend logic >= 90%
[ ] Frontend meaningful coverage >= 70%
[ ] Coverage thresholds automatically checked
[ ] Coverage HTML report generated
```

Documentation

```
[ ] README
[ ] Architecture explained
[ ] Makefile documented
[ ] Payment flow documented
[ ] Redis usage documented
[ ] Worker architecture documented
[ ] Test commands documented
[ ] Coverage documented
```

153. Başarı Kriteri

Bu görev başarılı sayılır eğer geliştirici:

1. Go ile HTTP backend oluşturabiliyorsa.
2. Kodunu mantıklı package'lara ayırabiliyorsa.
3. PostgreSQL ile ilişkili data modelleyebiliyorsa.
4. SQL query yazabiliyorsa.
5. Transaction kullanabiliyorsa.
6. Concurrent stock problemini anlayabiliyorsa.
7. Redis session kullanabiliyorsa.
8. Redis cache kullanabiliyorsa.
9. Cache invalidation yapabiliyorsa.
10. Redis queue oluşturabiliyorsa.
11. Worker geliştirebiliyorsa.
12. Email'i asynchronous gönderebiliyorsa.
13. Authentication yapabiliyorsa.
14. Authorization yapabiliyorsa.
15. Resource ownership kontrolü yapabiliyorsa.
16. Cart ve checkout geliştirebiliyorsa.
17. Order/payment ayrımını anlayabiliyorsa.
18. Manual bank transfer akışı geliştirebiliyorsa.
19. User payment submission ile admin approval farkını anlayabiliyorsa.
20. Admin approve/reject akışı geliştirebiliyorsa.
21. Payment state transition'larını backend'de koruyabiliyorsa.
22. React frontend geliştirebiliyorsa.
23. React'i Go binary içine embed edebiliyorsa.

24. Image processing worker geliştirebiliyorsa.
 25. FFmpeg video worker kullanabiliyorsa.
 26. Türkçe/İngilizce destek verebiliyorsa.
 27. Docker Compose ile sistemi çalıştırabiliyorsa.
 28. Kullanışlı Makefile yazabiliyorsa.
 29. Unit test yazabiliyorsa.
 30. Integration test yazabiliyorsa.
 31. Frontend component test yazabiliyorsa.
 32. Kritik full-stack flow için E2E test oluşturabiliyorsa.
 33. Backend'de en az %80 anlamlı coverage sağlayabiliyorsa.
 34. Kritik business logic'te %90 civarı coverage sağlayabiliyorsa.
 35. Test yazmayı yalnızca coverage rakamı için değil gerçek davranışı doğrulamak için kullanabiliyorsa.
-

Final Requirement

Yeni bir developer repository'yi clone ettikten sonra normal olarak şu şekilde başlayabilmelidir:

```
git clone <repository>
cd ecommerce

make up
make migrate
make create-admin
```

Application:

```
http://localhost:8080
```

Mailpit:

```
http://localhost:8025
```

Tests:

```
make test
```

Coverage:

```
make coverage
```

CI-style local validation:

```
make ci
```

Application:

- Go backend kullanmalıdır.
- PostgreSQL kullanmalıdır.
- Redis kullanmalıdır.
- React frontend Go binary içerisinde servis edilmelidir.
- Customer/Admin authorization bulunmalıdır.
- Manual bank transfer payment bulunmalıdır.
- Customer transfer yaptığını bildirebilmelidir.
- Admin payment'i approve veya reject edebilmelidir.
- Payment durumları email ile kullanıcıya bildirilmelidir.
- Email'ler Redis queue üzerinden gönderilmelidir.
- Image/video optimization background worker üzerinden yapılmalıdır.
- Türkçe ve İngilizce desteklenmelidir.
- Makefile aktif olarak kullanılmalıdır.
- Critical business logic automated tests ile korunmalıdır.
- Backend meaningful code coverage minimum %80 hedefini karşılamalıdır.
- Repository'nin normal development workflow'u Makefile üzerinden yürütülebilmelidir.